

## **EXPERIMENT-2**

**PROBLEM STATEMENT:** Installation and setup of WordNet libraries.

### **THEORY:**

WordNet is a large, manually curated lexical database developed at Princeton University. Unlike traditional dictionaries that list words alphabetically, WordNet organizes words based on their meanings. Words with similar meanings are grouped into sets called synsets (synonym sets). Each synset represents a single distinct concept and is associated with a short definition (gloss) and example usage. A word may belong to multiple synsets, reflecting the fact that many English words have more than one meaning.

WordNet categorizes words into four major lexical classes: nouns, verbs, adjectives, and adverbs. These word classes are interconnected through various semantic relationships, which help in understanding the meaning and context of words. Some of the important semantic relationships in WordNet include:

- **Synonymy:** Words with similar meanings
- **Antonymy:** Words with opposite meanings
- **Hypernymy:** A more general term
- **Hyponymy:** A more specific term
- **Meronymy:** Part–whole relationship

WordNet is widely used in many real-world applications such as search engines, machine translation systems, chatbots, sentiment analysis, text summarization, and question-answering systems. By providing structured semantic information, WordNet helps computers understand not just the surface form of words but also their underlying meanings and relationships.

Thus, the WordNet library plays a crucial role in Natural Language Processing by offering a comprehensive and structured representation of lexical knowledge. This experiment focuses on installing and using the WordNet library in Python to explore word meanings and demonstrate its usefulness in NLP-based applications.

## **CODE AND OUTPUT:**

### **Step 1: Install NLTK Library**

The Natural Language Toolkit (NLTK) library is installed using the pip command:

---

```
pip install nltk
```

---

```
Requirement already satisfied: nltk in /usr/local/lib/python3.12/dist-packages (3.9.1)  
Requirement already satisfied: click in /usr/local/lib/python3.12/dist-packages (from nltk) (8.3.1)  
Requirement already satisfied: joblib in /usr/local/lib/python3.12/dist-packages (from nltk) (1.5.3)  
Requirement already satisfied: regex<=2021.8.3 in /usr/local/lib/python3.12/dist-packages (from nltk) (2025.11.3)  
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from nltk) (4.67.1)
```

---

### **Step 2: Import NLTK and Download WordNet Corpus**

Import the NLTK library and download the WordNet corpus

---

```
import nltk  
nltk.download('wordnet')
```

---

```
[nltk_data] Downloading package wordnet to /root/nltk_data...  
True
```

---

### **Step 3 : Import WordNet Module**

The WordNet corpus is imported from the NLTK library:

```
from nltk.corpus import wordnet as wn
```

### **Step 4: Retrieve Synsets for a Given Word**

The synsets for the word “computer” are retrieved using the `synsets()` function:

```
synsets = wn.synsets("computer")  
print(synsets)
```

---

```
[Synset('computer.n.01'), Synset('calculator.n.01')]
```

---

### **Step 5: Find Antonyms of the Word "good"**

Antonyms of the word **good** are extracted by iterating through synsets and lemmas:

```
antonyms = set()
for syn in wn.synsets("good"):
    for lemma in syn.lemmas():
        if lemma.antonyms():
            antonyms.add(lemma.antonyms()[0].name())

print(antonyms)
```

---

```
{'evilness', 'ill', 'bad', 'badness', 'evil'}
```

### Step 6: Find Synonyms of the Word "happy"

Synonyms of the word **happy** are obtained using WordNet synsets:

```
synonyms = set()
for syn in wn.synsets("happy"):
    for lemma in syn.lemmas():
        synonyms.add(lemma.name())

print(synonyms)
```

---

```
{'glad', 'happy', 'felicitous', 'well-chosen'}
```

### LEARNING OUTCOME:

Through this program, I learn about WordNet libraries and their applications such as search engines, machine translation systems, chatbots, sentiment analysis, text summarization, and question-answering systems

## **EXPERIMENT-3**

**PROBLEM STATEMENT:** Perform text tokenization using NLTK.

### **THEORY:**

Text tokenization is a fundamental concept in Natural Language Processing (NLP) that deals with dividing a large body of text into smaller, meaningful units known as tokens. These tokens form the basic building blocks for further text analysis and language understanding tasks. Since natural language text is unstructured, tokenization helps convert it into a structured format that can be easily processed by computational systems.

A token is the smallest unit of text that carries meaning. Tokens can represent sentences, words, numbers, punctuation marks, or symbols depending on the level of analysis required.

Tokenization is usually performed at two primary levels:

**Sentence tokenization:** It refers to the process of splitting a paragraph or document into individual sentences. This is important for understanding sentence boundaries and is commonly used in text summarization, document classification, and question-answering systems.

**Word tokenization:** It involves breaking sentences into individual words and punctuation symbols. It plays a crucial role in tasks such as frequency analysis, sentiment analysis, part-of-speech tagging, and information retrieval.

**Tokenization must handle various challenges present in natural language, such as:**

- Punctuation marks
- Abbreviations
- Contractions (e.g., *don't*, *can't*)
- Numbers and special characters
- Multi-word expressions

Another important concept related to tokenization is lexical analysis, which refers to the conversion of a sequence of characters into a sequence of tokens. Tokenization acts as the first step in lexical analysis and sets the foundation for higher-level linguistic processing.

Tokenization directly affects the performance of downstream NLP tasks. Poor tokenization may lead to loss of meaning or incorrect interpretation of text. Therefore, accurate tokenization is essential for ensuring reliable results in text processing systems.

## CODE AND OUTPUT:

```
pip install nltk
```

```
Requirement already satisfied: nltk in /usr/local/lib/python3.12/dist-packages (3.9.1)  
Requirement already satisfied: click in /usr/local/lib/python3.12/dist-packages (from nltk) (8.3.1)  
Requirement already satisfied: joblib in /usr/local/lib/python3.12/dist-packages (from nltk) (1.5.3)  
Requirement already satisfied: regex<=2021.8.3 in /usr/local/lib/python3.12/dist-packages (from nltk) (2025.11.3)  
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from nltk) (4.67.1)
```

```
import nltk  
nltk.download('punkt_tab')
```

```
[nltk_data] Downloading package punkt_tab to /root/nltk_data...  
[nltk_data] Unzipping tokenizers/punkt_tab.zip.  
True
```

```
text = """Natural Language Processing (NLP) is a rapidly growing field of Artificial Intelligence."""
```

```
from nltk.tokenize import sent_tokenize, word_tokenize
```

```
word_tokens = word_tokenize(text)  
print(word_tokens)
```

```
['Natural', 'Language', 'Processing', '(', 'NLP', ')', 'is', 'a', 'rapidly', 'growing', 'field', 'of', 'Artificial', 'Intelligence', '.']
```